Programozás II.

2. labor

Szekeres György

2026. március 7.

Tartalom

1. Előző órán...
2. Konstansok
3. Tömbök
4. Random osztály
5. String
6. Függvények
7. Gyakorlás

Előző órán...

Az előző órán tanultuk (átismételtük) a(z)

- azonosítók elnevezését;
- változók definiálását;
- bekérést és kiíratást;
- konvertálást és castolást;
- elágazásokat (if-else, switch-case);
- ciklusokat (while, do-while, for, foreach);
- 1D tömböket.

Konstansok

Létrehozhatunk olyan változókat, amelyek értéke **állandó** a program futása során. Lehet konstansból tömböt is létrehozni, és mindig csak a megfelelő méretű memóriát foglalja le. Szintaxisa:

```
1 const típus változóNeve = érték;
```

vagy

```
1 const típus[] változóNeve = értékek;
```

Tömbök

Tömbök, vektorok

- Az 1D tömböket vektornak nevezzük
- Az nD tömböket mátrixnak nevezzük, ahol n>1
- A már deklarált tömbök méretét **nem** lehet megváltoztatni
- Méretek lekérdezése:
  - Vektor: név.Length;
  - Mátrix: név.GetLength(dimenzióSzáma);

Mátrixok

- Van szélessége és magassága
- Szintaxisa:

```
típus[,] változóNeve = new típus[magasság, szélesség];
```

- Pl: tomb.GetLength(0); –> Magasság
- Pl: tomb.GetLength(1); –> Szélesség

Példa a használatra

- Készíts egy programot, amely egy 15 elemű tömbben eltárolja a háromszögszámokat! (A háromszögszámok olyan természetes számok, amelyek az első n darab egymást követő egész szám összegeként állíthatók elő. Nevüket onnan kapták, hogy ennyi pontot (kavicsot) szabályos háromszög alakban lehet elrendezni.
  - Mi a 14. elem?
  - Milyen hosszú a tömb?
  - Írd ki a páros indexű elemeket!
- Készíts egy 8x8-as sakktáblát! A fehér mezőt jelölje az 1, a feketét, pedig a 0.
  - Hány fehér és hány fekete mezőt tartalmaz a tábla?
  - Kérdezd le a magasságot és a szélességet is!
  - Írd ki a 3. sor elemeit!

Random osztály

- Ha random értéket szeretnénk generálni, akkor használjuk
- A System.Random osztályt fogjuk használni
- Példányosítani kell:

```
Random példányNeve = new Random();
```

- A *new* segítségével hozzuk létre az új példányt a Random osztályból
- Ezután a példány nevével tudjuk használni a Random osztály tagfüggvényeit
- Random szám generálásához a Next() függvényt kell használni
- Ha nem adunk neki paramétert, akkor tetszőleges *int* kerül generálásra
- Megadhatunk tól-ig határt is: Next(tól, ig);

```
Console.WriteLine(példányNeve.Next());
Console.WriteLine(példányNeve.Next(0,11)); //0-10 között lehet, a 11 nem
```

Random osztály

- Lebegőpontos számokhoz a NextDouble() alkalmas
- Nem lehet paramétert megadni
- 0.0 és 1.0 közötti értékeket generál
- Érdekesség: NextBytes(); a byte tömbök feltöltésére szolgál
- Ha mondjuk 0 és 100 közötti lebegőpontos számokat szeretnénk:

```
1 Random r = new Random();
2 double randomSzam = r.NextDouble() * 100;
```

Példa a használatra

- Készíts egy 10x10-es mátrixot. Töltsd fel random számokkal 1-100 között.
  - Add meg a legnagyobb elem értékét, és pozícióját.
  - Mennyi a 9. sor átlaga?
  - Hogyan lehet 0-5 közötti lebegőpontos számokat generálni?

String

- A string valójában egy char tömbként értelmezhető
- Szemben a tömbökkel a string mérete változtatható
- Példa erre:

```
1 char[] targyT = new char[] {'P', 'r', 'o', 'g', 'r', 'a', 'm', 'o', 'z',
2 'á', 's', 'I', 'I'};
3 string targy = "Programozás II";
```

- A string karaktereit ugyanúgy elérhetjük, mint a tömbök elemeit, azaz indexelve
- A string hossza a név.Length; segítségével kérhető le

String

A String osztály metódusai:

- Másolás: .Substring(honnan); vagy .Substring(honnan, mennyit)
- Keresés (index): .IndexOf(keresett string) vagy .IndexOf(keresett string, honnan); Ha nincs találat, -1 az eredmény, egyébként a kezdőindex.
- Csere: .Replace(régi, új); A régi stringet lecseréli az újra a megadott stringben.
- Kisbetű: .ToLower(); Kisbetűssé alakítja a stringet.
- Nagybetű: .ToUpper(); Nagybetűssé alakítja a stringet.
- Darabolás: .Split(határoló karakter); A megadott határoló alapján feldarabolja a stringet, és egy tömböt ad eredményül.

String

A String osztály metódusai:

- Láthatatlan karakterek: .Trim(); Eltávolítja a láthatatlan karaktereket (escape) a string elejéről és végéről.
- Törlés: .Remove(honnan, mennyit); Törli a stringből az adott indextől a megadott számú karaktert.
- Tartalmazás: .Constains(string); Bool értéket ad vissza, hogy a string tartalmazza-e a megadott stringet.
- Beszúrás: .Insert(hova, mit); A megadott indexre beszúrja a stringet.
- Feltölt: .PadLeft(meddig, mivel); vagy .PadRight(meddig, mivel); A string elé vagy után a megadott karakterszámban kiegészíti a megadott karakterrel.

String

Escape karakterek:

- \a: alert hang
- \b: törlés, backspace
- \n: új sor
- \r: visszaugrik a kurzor
- \t: vízszintes tabulátor
- \v: függőleges tabulátor
- \': ' karaktert illeszt be
- \": " karaktert illeszt be
- \\: \karaktert illeszt be
- @ esetén szó szerinti stringként jelenik meg

Példa a használatra

- Adott az alábbi string: "A gépi látás területén ma már meghatározóak a gépi tanulás alapú Deep Learning megoldások, azokon belül is a Convolutional Neural Network és Vision Transformer modellek. Egy-egy konkrét osztályozási vagy objektumdetekciós feladatra a megfelelő modell megtalálása és feltanítása fontos feladat. A kutatási feladatban a részfeladatok: különböző CNN, ViT, Transfer Learning és AutoML módszer megismerésére, tanító adathalmaz gyűjtése és címkézése, modellek feltanítása és kiértékelése."
  - Add meg a "gépi látás" string pozícióját.
  - Hány karakterből áll a megadott string?
  - Cseréld ki a CNN-t a megfelelő fogalomra.
  - Darabold fel a stringet a szóközök mentén! Mekkora tömböt kapunk így?

Függvények

- Lényege a többszöri felhasználhatóság
- Minden fgv-nek megvan a maga szerepe, és a ráutaló neve
- Kétféle fő alprogramot különböztetünk meg:
  - Eljárások (void –> Nincs visszatérési értéke)
  - Függvények (return ág –> Minden esetben kötelező)
- A return utáni kódsorok **nem** futnak le
- Önmagukat is meghívhatják (**rekurzió**), ezt le kell kezelni, a végtelen ciklus ellen, különben **System.StackOverflowException** hibát kapunk
- Szintaxisa:

```
1 static típus név([paraméter(ek)])
2 {
3     utsaítás(ok);
4     [return valami;] //Ha függvény!
5 }
```

Függvények

Paraméterezés lehet: Érték szerinti paraméter átadás

- Az értékek átmásolódnak a formális paraméterbe
- A formális paraméter normál változóként kezelhető

```
1 static void Main(string[] args){
2     int a = 6, b = 4, c;
3     c = Terület(a, b);
4     Console.WriteLine($"A téglalap területe: {c}");
5 }
6 static int Terület(int szam1, int szam2) {
7     return szam1 * szam2;
8 }
```

Függvények

Paraméterezés lehet: Cím (referencia) szerinti paraméter átadás

- **ref** kulcsszó használata a meghíváskor, és a fgv megírásakor is
- Ilyenkor hivatkozás jön létre a memória tartományára, csak a memóriaterület címe/azonosítója adódik át, így az alprogramon belül végzett változtatások érvénybe lépnek az átadott változóban is.
- **Fontos**: az összetett adatszerkezetek, tömbök, objektumok minden esetben referencia, tehát cím szerint adódnak át!

```
1 static void Main(string[] args){
2     int a = 6, b = 4, c;
3     c = Terület(ref a, ref b);
4     Console.WriteLine($"A téglalap területe: {c}");
5 }
6 static int Terület(ref int szam1, ref int szam2) {
7     return szam1 * szam2;
8 }
```

Függvények

Paraméterezés lehet: Kimeneti változó

- **out** kulcsszó használata a kimeneti változó esetén
- Kötelező az értékadás ilyenkor!
- Kezdetben a változónak/paraméternek nincs értéke

```
1 static void Main(string[] args){
2     int a = 6, b = 4, c;
3     Terület(a, b, out c);
4     Console.WriteLine($"A téglalap területe: {c}");
5 }
6 static int Terület(int szam1, int szam2, out int eredmeny) {
7     eredmeny = szam1 * szam2;
8 }
```

Függvények

Túlterhelés

- Azonos elnevezés, más paraméterezés
- Célja, hogy ugyanazon megvalósításhoz ne kelljen több fgv, hanem ugyanazzal a névvel tudjuk meghívni
- Ezt fgv túlterhelésnek nevezzük

```
static int Terület(int szam1, int szam2) {
    return szam1 * szam2;
}
static double Terület(int szam1, double szam2) {
    return szam1 * szam2;
}
static double Terület(double szam1, int szam2) {
    return szam1 * szam2;
}
static double Terület(double szam1, double szam2) {
    return szam1 * szam2;
}
```

Függvények

Tulajdonságok dokumentálása

- Azonos elnevezés, más paraméterezés
- Célja, hogy ugyanazon megvalósításhoz ne kelljen több fgv, hanem ugyanazzal a névvel tudjuk meghívni

```
1 /// <summary> Ez a függvény kiszámítja a téglalap területét</summary>
2 /// <param name="szam1">Téglalap egyik oldala</param>
3 /// <param name="szam2">Téglalap másik oldala</param>
4 /// <returns>A kiszámított terület értéke</returns>
5 /// <exception cref="System.OverflowException"/>
6 static double Terület(double szam1, double szam2) {
7     return szam1 * szam2;
8 }
```

Függvények

Rekurzió

- Önmagát hívja meg
- Kölcsönös rekurzió: A() meghívja B()-t, majd B() meghívja A()-t
- Nagyon erőforrás igényes
- Kezelni kell a végtelenciklus kialakulását!

```
1 static ulong Faktorialis(ulong n){
2     if (n <= 1)
3         return n;
4     return n * Faktorialis(n - 1);
5 }
```

Példa a használatra

- Készíts egy eljárást, amely egy stringben megkeresi az email címet! Ha megtalálja azt, írd ki azt a konzolra, vagy ha nincs, akkor "Elérhetőség nem található!".
- Készíts egy függvényt, amely megadja, hogy a string tartalmaz-e telefonszámot. Ha igen, írja ki, hogy "A bemenet tartalmaz elérhetőséget!", ellenkező esetben ennek a tagadását.

Gyakorlás

- **1. Háromszögszámok elemző** (Tömb + Függvény)
  20 elemű tömb első 20 háromszögszámmal. Páratlan indexek összege, páratlan számok vektora.
- **2. Sakk tábla statisztika** (2D + Random)
  10x12 mátrix: 0=fehér(35%), 1=fekete(35%), 2=király(20%), 3=huszár(10%). Statisztika, pozíciók.
- **3. Szöveg statisztika mester** (String + Random)
  1000 karakter random szöveg: szavak, mássalhangzók, palindrom szavak számolása.

Gyakorlás

Kötelező minden feladatban:

- Legalább 2 fgv (1 void, 1 visszatérésű)
- String kezelés (Split, Substring)
- Tömb/Random használat
- Ternáris operátor (? :)